



Universidade Federal do Amazonas
Instituto de Ciências Exatas e Tecnologia
Bacharelado em Engenharia de Software



Felipe William Galdino da Silva,
João Carlos G. Iannuzzi,
Reyner C. Silva Alegria

Relatório Técnico - Video Party P2P

Relatório Técnico

Itacoatiara - AM
2026

Felipe William Galdino da Silva,
João Carlos G. Iannuzzi,
Reyner C. Silva Alegria

Relatório Técnico - Video Party P2P

Trabalho apresentado ao professor Antonio Alberto Sena dos Santos, do Instituto de Ciências Exatas e Tecnologia da Universidade Federal do Amazonas, como atividade prática para a obtenção de nota na disciplina de Sistemas Distribuídos.

Itacoatiara - AM

2026

SUMÁRIO

Sumário	2
1 Introdução	4
2 Objetivos	4
3 Fundamentação Teórica	5
3.1 Arquitetura P2P híbrida	5
3.2 Comunicação TCP e mensagens JSON	5
3.3 Integridade, hash e cifra didática	6
4 Arquitetura da Solução	6
4.1 Fluxo geral	7
5 Protocolo Implementado	8
6 Metodologia de Desenvolvimento	9
7 Resultados	10
8 Validação	11
9 Discussão	12
10 Limitações e Trabalhos Futuros	13
11 Conclusão	13
Referências Bibliográficas	15

RESUMO

Este relatório apresenta o desenvolvimento do Video Party P2P, uma aplicação desktop criada em Flutter para demonstrar conceitos de sistemas distribuídos por meio de uma festa colaborativa de vídeos. A solução implementa uma arquitetura P2P híbrida: um tracker TCP centraliza metadados, peers e a playlist global, enquanto os arquivos são transferidos diretamente entre clientes. Cada cliente também atua como servidor de upload TCP, permitindo que vídeos sejam anunciados, localizados, baixados em partes paralelas, validados por SHA-256 e reproduzidos localmente. O sistema ainda inclui transferência cifrada didática, fallback por relay no tracker, prefetch do próximo item da playlist e interface gráfica para operação em rede local ou via Tailscale.

Palavras-chave: sistemas distribuídos; peer-to-peer; TCP; Flutter; transferência de arquivos.

1 INTRODUÇÃO

Sistemas distribuídos são formados por componentes independentes que se comunicam por rede para oferecer ao usuário a impressão de um sistema único (TANENBAUM; STEEN, 2017). Esse modelo aparece em aplicações de compartilhamento de arquivos, streaming, armazenamento replicado, jogos, mensageria e diversas arquiteturas modernas de computação. Entre as organizações possíveis, redes *peer-to-peer* (P2P) se destacam por distribuir responsabilidades entre os próprios clientes, reduzindo a dependência de um servidor único para transferência de dados (COULOURIS et al., 2012).

O trabalho desenvolvido neste repositório, denominado **Video Party P2P**, aplica esses conceitos em um cenário prático: uma playlist colaborativa de vídeos. A proposta é permitir que diferentes máquinas compartilhem arquivos de vídeo em uma rede, adicionem itens a uma playlist global e reproduzam esses vídeos localmente após o download. Para simplificar descoberta e coordenação, foi adotada uma arquitetura P2P híbrida, na qual um tracker central registra metadados e peers, mas a transferência de arquivos ocorre preferencialmente de peer para peer.

O projeto foi implementado em Flutter e Dart, utilizando sockets TCP da biblioteca `dart:io`, cálculo de hashes com `crypto` e reprodução multimídia com `media_kit`. Como resultado, o sistema demonstra comunicação por mensagens, descoberta de recursos, transferência direta, paralelismo, verificação de integridade, tolerância parcial a falhas por relay e atualização dinâmica de estado na interface.

2 OBJETIVOS

O objetivo geral do trabalho foi construir uma aplicação funcional que materializasse conceitos de sistemas distribuídos em uma experiência visual e testável. Em vez de apenas simular mensagens, o sistema deveria transferir arquivos reais entre máquinas e permitir a reprodução local dos vídeos baixados.

Os objetivos específicos foram:

- implementar um tracker TCP para registro de peers, vídeos e playlist global;
- permitir que cada cliente atue também como servidor de upload;
- localizar arquivos por hash e por nome, mantendo compatibilidade com a ideia de comando `LOOKUP <nome>`;

- realizar downloads diretamente entre peers, com divisão do arquivo em até quatro partes paralelas;
- aplicar uma cifra simples baseada em XOR e SHA-256 com finalidade didática;
- validar a integridade do arquivo baixado por meio de hash SHA-256;
- oferecer relay pelo tracker quando a conexão direta entre peers falhar;
- disponibilizar uma interface gráfica para configuração da rede, playlist, downloads e reprodução.

3 FUNDAMENTAÇÃO TEÓRICA

3.1 Arquitetura P2P híbrida

Em uma arquitetura cliente-servidor pura, o servidor concentra o armazenamento e a entrega dos dados. Em uma arquitetura P2P, os clientes também fornecem recursos diretamente uns aos outros. O Video Party P2P adota uma organização híbrida: o tracker funciona como ponto conhecido de coordenação, mas não é o caminho principal dos arquivos. Essa escolha reduz a complexidade de descoberta e preserva a característica distribuída da transferência.

O tracker mantém uma visão global dos vídeos disponíveis, dos peers conectados e da playlist. Os peers mantêm suas bibliotecas locais, calculam hashes dos arquivos, anunciam sua disponibilidade e atendem solicitações de download. Assim, o sistema separa controle e dados: mensagens de controle passam pelo tracker; bytes de vídeo trafegam diretamente entre peers sempre que possível.

3.2 Comunicação TCP e mensagens JSON

A comunicação foi implementada sobre TCP, protocolo que oferece fluxo confiável, ordenado e orientado a conexão (EDDY, 2022). Cada comando do sistema é enviado como uma linha JSON terminada por `\n`. Esse formato facilita a interoperabilidade e a depuração, pois cada mensagem pode ser lida e interpretada como uma estrutura textual clara.

No tracker, comandos como `REGISTER`, `LIST`, `LOOKUP`, `DOWNLOAD`, `ADD_PLAYLIST` e `RELAY_DOWNLOAD` representam as operações de controle. Nos peers, o comando principal é `GET_FILE`, usado para solicitar um intervalo específico de bytes de um arquivo.

3.3 Integridade, hash e cifra didática

Cada vídeo é identificado por seu hash SHA-256, calculado sobre o conteúdo completo do arquivo. Esse identificador evita depender apenas do nome do arquivo e permite verificar se o arquivo recebido é exatamente igual ao anunciado. Ao final de cada download, o cliente recalcula o SHA-256 do arquivo remontado e compara o resultado com o hash registrado no tracker (National Institute of Standards and Technology, 2015).

O tráfego entre peers utiliza uma cifra simples chamada `xor-sha256-demo`. Ela combina XOR com blocos derivados de SHA-256 a partir de uma chave fixa do projeto. Essa abordagem não deve ser tratada como criptografia de produção, mas cumpre uma função didática: mostrar que a transferência pode aplicar transformações sobre os bytes e desfazê-las no receptor mantendo a verificação de integridade.

4 ARQUITETURA DA SOLUÇÃO

A implementação foi organizada em cinco módulos principais:

- `models.dart`: define os modelos de domínio, como vídeo, peer, entrada de playlist, snapshot do tracker e progresso de download;
- `tracker_server.dart`: implementa o servidor TCP responsável por registro, listagem, busca, playlist e relay;
- `peer_node.dart`: implementa o servidor de upload do peer, varredura de pasta, download direto, download por relay, cifra e validação de hash;
- `app_controller.dart`: coordena estado da aplicação, conexões, downloads, player, prefetch e notificações para a interface;
- `main.dart`: implementa a interface Flutter com painel de configuração, catálogo global, playlist, player, eventos e seleção de tema.

Tabela 1 – Responsabilidades dos componentes da aplicação

Componente	Responsabilidade
Tracker	Manter metadados globais, responder consultas, registrar peers e atuar como relay quando necessário.
Peer	Escanear vídeos locais, calcular hashes, servir uploads e baixar arquivos de outros peers.
Cliente Flutter	Exibir estado da rede, controlar configurações, reproduzir vídeos e acompanhar progresso de downloads.
Playlist global	Representar a fila colaborativa compartilhada entre os participantes.

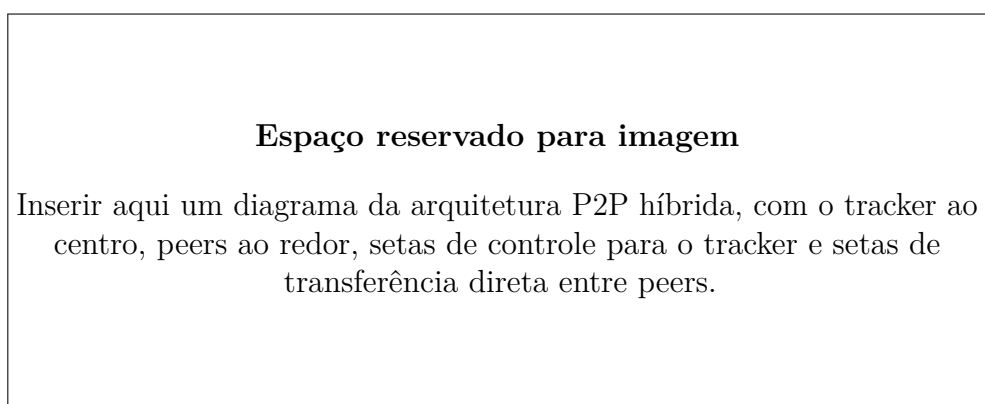


Figura 1 – Área reservada para o diagrama da arquitetura geral do Video Party P2P.

4.1 Fluxo geral

O fluxo de uso previsto envolve duas ou mais máquinas. Uma delas inicia o tracker e ativa também seu servidor de upload. Cada máquina informa seu IP anunciado, porta de upload e pasta local de vídeos. Ao escanear a pasta, o peer calcula o SHA-256 dos arquivos suportados e registra esses metadados no tracker. Depois disso, qualquer cliente pode atualizar a rede, visualizar o catálogo global, adicionar vídeos à playlist e reproduzir um item.

Quando um vídeo da playlist não está disponível localmente, o cliente consulta o tracker para descobrir quais peers possuem aquele hash. Em seguida, tenta baixar o arquivo diretamente dos peers encontrados. Se a conexão direta falhar, o cliente solicita ao tracker um `RELAY_DOWNLOAD`; nesse caso, o tracker conecta em um peer dono do arquivo e repassa os bytes ao solicitante.

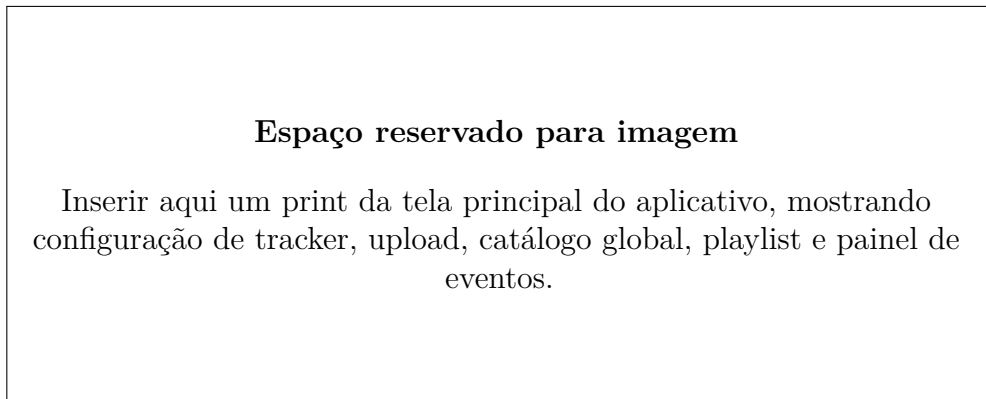


Figura 2 – Área reservada para a interface principal do aplicativo em execução.

5 PROTOCOLO IMPLEMENTADO

O protocolo usa mensagens JSON sobre TCP. A Listagem 1 mostra o registro de um peer com seus vídeos locais.

Listing 1 – Mensagem REGISTER enviada ao tracker.

```
1 {  
2   "type": "REGISTER",  
3   "peer": {  
4     "id": "peer-1",  
5     "name": "DJ Video",  
6     "host": "100.x.x.x",  
7     "port": 5051  
8   },  
9   "videos": [  
10    {"name": "show.mp4", "hash": "sha256", "size": 104857600}  
11  ]  
12 }
```

A busca pode ocorrer por hash ou por nome. O suporte à busca por nome aproxima o protocolo JSON da especificação textual LOOKUP <nome>, preservando legibilidade e estrutura.

Listing 2 – Consulta por nome no tracker.

```
1 {"type": "LOOKUP", "name": "show.mp4"}
```

Para transferência direta, o peer solicitante envia GET_FILE ao peer dono do arquivo, informando hash, deslocamento inicial e tamanho do intervalo desejado. Essa divisão permite downloads em partes paralelas.

Listing 3 – Solicitação de intervalo de arquivo a um peer.

```

1 {
2   "type": "GET_FILE",
3   "hash": "sha256",
4   "offset": 0,
5   "length": 1048576,
6   "encrypted": true
7 }

```

Caso a conexão direta não seja possível, o cliente usa o relay do tracker:

Listing 4 – Solicitação de relay ao tracker.

```

1 {
2   "type": "RELAY_DOWNLOAD",
3   "hash": "sha256",
4   "requesterId": "peer-2",
5   "offset": 0,
6   "length": 1048576,
7   "encrypted": true
8 }

```

Espaço reservado para imagem

Inserir aqui um diagrama de sequência com os passos REGISTER, LOOKUP, GET_FILE e RELAY_DOWNLOAD entre peer solicitante, tracker e peer dono do arquivo.

Figura 3 – Área reservada para o diagrama de sequência do protocolo de comunicação.

6 METODOLOGIA DE DESENVOLVIMENTO

O desenvolvimento foi conduzido de forma incremental. Primeiro foram definidos os modelos de domínio e o protocolo mínimo para registro e listagem. Em seguida, o tracker foi implementado como servidor TCP capaz de receber uma linha JSON, processar o comando e devolver uma resposta também em JSON.

Depois, foi implementado o peer. Cada peer passou a escanear uma pasta local, filtrar extensões de vídeo, calcular SHA-256 e iniciar um servidor TCP de upload. A transferência direta foi então evoluída para aceitar intervalos de bytes, permitindo

que o cliente dividisse um arquivo em partes. No código, o limite adotado foi de quatro partes paralelas, equilibrando simplicidade e demonstração de concorrência.

Com a transferência direta pronta, foi adicionada a validação de integridade. Após baixar todas as partes, o cliente concatena os arquivos temporários, recalcula o hash do arquivo final e remove a saída caso o hash não corresponda ao esperado. Essa etapa é importante porque transforma o tracker em fonte de verdade para metadados e impede que uma transferência corrompida seja aceita.

Em seguida, foi implementado o relay pelo tracker. Esse mecanismo atende cenários em que um peer não consegue conectar diretamente em outro, por exemplo devido a restrições de NAT ou firewall. O relay não substitui o P2P como caminho principal, mas aumenta a robustez do sistema.

Por fim, a interface Flutter foi criada para tornar a experiência operacional. Ela permite iniciar tracker, ativar upload, registrar vídeos, atualizar catálogo, montar playlist, acompanhar downloads, reproduzir vídeo localmente e alternar entre temas (Flutter, 2026; Dart, 2026; media_kit, 2026).

7 RESULTADOS

O resultado do trabalho é um aplicativo desktop funcional para Windows e Linux, com suporte a rede entre máquinas. Os principais comportamentos implementados foram:

- inicialização de tracker local em 0.0.0.0:4040;
- inicialização de servidor de upload em cada peer;
- registro de arquivos de vídeo com nome, tamanho e hash SHA-256;
- catálogo global consolidado pelo tracker;
- playlist global compartilhada;
- download direto peer-to-peer com tráfego cifrado;
- download paralelo por intervalos de bytes;
- validação de integridade após a remontagem;
- relay pelo tracker como alternativa;
- prefetch automático do próximo vídeo da playlist;

- reprodução local usando `media_kit`;
- interface responsiva com painel principal, barra lateral, painel de playlist, peers e eventos.



Figura 4 – Área reservada para evidência visual de download, playlist e reprodução local.

Tabela 2 – Resumo dos comandos do protocolo

Comando	Função
REGISTER	Registra peer e lista de vídeos disponíveis.
LIST	Retorna snapshot com vídeos, peers e playlist.
LOOKUP	Busca vídeo por hash ou nome e retorna peers donos.
DOWNLOAD	Alias de busca por hash usado antes do download.
ADD_PLAYLIST	Adiciona um vídeo registrado à playlist global.
GET_FILE	Solicita a um peer um intervalo de bytes do arquivo.
RELAY_DOWNLOAD	Solicita ao tracker o repasse dos bytes quando o acesso direto falha.

8 VALIDAÇÃO

A validação automatizada foi implementada em testes Flutter. O teste `p2p_transfer_test.dart` cobre dois cenários centrais. No primeiro, dois peers possuem o mesmo arquivo e um terceiro realiza download em partes paralelas, verificando se os bytes finais são idênticos aos bytes originais. No segundo, um tracker e um peer owner são iniciados localmente; o cliente realiza `LOOKUP` por nome e depois baixa o arquivo por relay do tracker.

Esses testes exercitam os pontos mais sensíveis do sistema: sockets TCP, varredura de pasta, hash SHA-256, transferência cifrada, remontagem de partes, consulta por nome e relay. Além disso, o projeto inclui um teste de interface que

verifica elementos essenciais da tela inicial, como o botão para iniciar o tracker e o ícone de downloads.

Os comandos previstos para verificação do projeto são:

Listing 5 – Comandos de verificação do projeto.

```
1 flutter analyze
2 flutter test
```

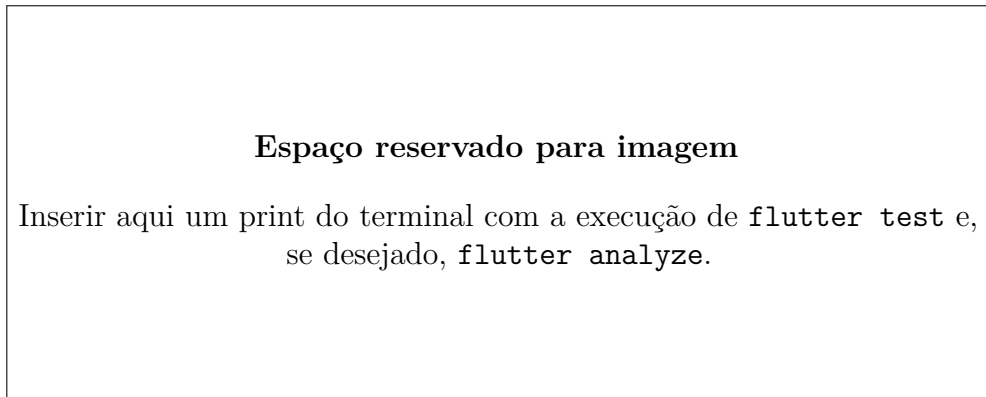


Figura 5 – Área reservada para evidência visual da validação automatizada.

9 DISCUSSÃO

A arquitetura híbrida mostrou-se adequada para o objetivo educacional do trabalho. O tracker simplifica descoberta e estado global, enquanto os peers preservam a distribuição da transferência de arquivos. Essa separação facilita a compreensão dos papéis do sistema: coordenação centralizada para metadados e comunicação distribuída para dados.

O download por partes evidencia o uso de paralelismo no cliente. Mesmo quando há poucos peers, a implementação divide o arquivo em intervalos e usa `Future.wait` para executar as transferências de forma concorrente. Quando existem múltiplos owners, as partes podem ser distribuídas entre eles. A validação por SHA-256 garante que essa remontagem não altere o conteúdo final.

O relay pelo tracker representa uma decisão de robustez. Em redes reais, conexões diretas podem falhar por NAT, firewall, IP anunciado incorretamente ou indisponibilidade temporária de um peer. Ao permitir que o tracker repasse os bytes, o sistema sacrifica parte da descentralização e aumenta o uso do servidor, mas melhora a chance de concluir o download.

A cifra `xor-sha256-demo` deve ser entendida como recurso demonstrativo. Ela mostra a ideia de tráfego transformado e reversível, mas não substitui protocolos criptográficos reais. Em uma versão de produção, o caminho adequado seria usar TLS, autenticação de peers, autorização de acesso e negociação segura de chaves.

10 LIMITAÇÕES E TRABALHOS FUTUROS

Apesar de funcional, o sistema ainda possui limitações. O tracker mantém estado apenas em memória, portanto os dados são perdidos quando ele é encerrado. Não há autenticação de usuários ou peers, e a cifra implementada é apenas didática. A playlist também não possui persistência, ordenação colaborativa avançada ou resolução de conflitos.

Como trabalhos futuros, destacam-se:

- persistir catálogo, peers recentes e playlist em banco local;
- adicionar TLS e autenticação entre tracker e peers;
- implementar descoberta automática em rede local;
- permitir remoção e reordenação colaborativa da playlist;
- medir desempenho com arquivos maiores, múltiplos peers e redes distintas;
- melhorar tolerância a falhas durante downloads parciais;
- criar empacotamento final para distribuição do aplicativo.

11 CONCLUSÃO

O Video Party P2P cumpriu o objetivo de demonstrar, de forma prática, conceitos importantes de sistemas distribuídos. A aplicação combina comunicação TCP, protocolo de mensagens, tracker, peers, transferência direta, relay, paralelismo, verificação de integridade e interface gráfica em um único sistema funcional.

A solução evidencia como uma arquitetura P2P híbrida pode equilibrar simplicidade de coordenação e distribuição de carga. O tracker organiza a rede e a playlist, mas os peers continuam responsáveis por armazenar e entregar os vídeos. A validação por SHA-256 e os testes automatizados reforçam a confiabilidade da transferência, enquanto o `fetch` e o player integrado aproximam o protótipo de uma experiência real de uso.

Assim, o trabalho vai além de uma prova de conceito isolada: ele materializa os pilares da disciplina em um aplicativo executável, observável e extensível, servindo como base para discussões sobre escalabilidade, confiabilidade, segurança e projeto de protocolos distribuídos.

REFERÊNCIAS BIBLIOGRÁFICAS

COULOURIS, G. et al. **Distributed Systems: Concepts and Design**. 5. ed. [S.l.]: Addison-Wesley, 2012.

Dart. **dart:io library**. 2026. <<https://api.dart.dev/dart-io/>>. Acesso em 3 maio 2026.

EDDY, W. **Transmission Control Protocol (TCP)**. 2022. <<https://www.rfc-editor.org/rfc/rfc9293>>. RFC 9293.

Flutter. **Flutter Documentation**. 2026. <<https://docs.flutter.dev/>>. Acesso em 3 maio 2026.

media_kit. **media_kit package**. 2026. <https://pub.dev/packages/media_kit>. Acesso em 3 maio 2026.

National Institute of Standards and Technology. **Secure Hash Standard (SHS)**. 2015. <<https://doi.org/10.6028/NIST.FIPS.180-4>>. FIPS PUB 180-4.

TANENBAUM, A. S.; STEEN, M. V. **Distributed Systems**. 3. ed. [S.l.]: Maarten van Steen, 2017.